



ADDIS ABABA
**SCIENCE AND
TECHNOLOGY**
UNIVERSITY
UNIVERSITY FOR INDUSTRY

**PREDICTING HIGH-RISK TECHNICAL DEBT IN
OPEN-SOURCE SOFTWARE PROJECTS USING
MACHINE LEARNING AND CODE METRICS**

A MASTER'S THESIS PROPOSAL

BY

ABDULMAJID AWOL SEID

ADVISOR: TESFAYE GIDEY (PhD)

DEPARTMENT SOFTWARE ENGINEERING

SCHOOL OF POST GRADUATE STUDIES

FEBRUARY 2026

Approval Page

**Title: Predicting High-Risk Technical Debt in Open-Source Software Projects Using
Machine Learning and Code Metrics**

Student Name: Abdulmajid Awol Seid Signature: _____ Date: _____

Approved by the examining committee members:

Name	Academic Rank	Signature	Date
Advisor: <u>Dr. Tesfaye Gidey</u>	_____	_____	_____
Co-Advisor: _____	_____	_____	_____
Examiner: _____	_____	_____	_____
Examiner: _____	_____	_____	_____
Chairperson: _____	_____	_____	_____

Name	Signature	Date
DGC Chairperson: _____	_____	_____
Dean/Vice Dean: _____	_____	_____
Dean for Graduate Programs: _____		

Approval Page.....	2
List of Tables.....	4
List of Figures.....	5
Abbreviation and acronyms.....	6
Predicting High-Risk Technical Debt in Open-Source Software Projects Using Machine Learning and Code Metrics.....	1
1. INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.2 Significance of the Study.....	2
1.3 Objectives.....	3
1.3.1 General Objective.....	3
1.3.2 Specific Objectives.....	3
1.3.3 Research Questions.....	4
2. RELATED LITERATURE REVIEW.....	4
2.1 Technical Debt Detection and Classification.....	4
2.2 Technical Debt Prediction Using Metrics and Project History.....	5
2.3 Technical Debt in Machine Learning and AI Oriented Systems.....	5
2.4 Summary of Gaps and Motivation for This Study.....	6
3. RESEARCH METHODOLOGY.....	7
3.1 Research Design Overview.....	7
3.2 Data Sources and Project Selection.....	8
3.3 Feature Extraction and Representation (Independent Variables).....	9
3.4 High Risk Technical Debt Labeling (Dependent Variable).....	11
3.5 Model Development, Training, and Validation.....	13
3.6 Tools, Feasibility, Risk Mitigation, and Expected Outcomes.....	13
4. WORK SCHEDULE AND COST BREAKDOWN.....	15
4.1 Work Schedule and Timeline.....	15
4.2 Budget Breakdown.....	16
REFERENCES.....	18

List of Tables

<u>Table 1. Candidate predictive features for high-risk technical debt prediction</u>	<u>11</u>
<u>Table 2. Tools Used in the Study</u>	<u>14</u>
<u>Table 3. Activity Breakdown</u>	<u>16</u>
<u>Table 4. Budget Breakdown</u>	<u>17</u>

List of Figures

<u>Figure 1. Thesis Work Schedule (Gantt Chart)</u>	<u>16</u>
---	-----------

Abbreviation and acronyms

AI	Artificial Intelligence
AUC	Area Under the Curve
CI/CD	Continuous Integration / Continuous Deployment
ETB	Ethiopian Birr
LOC	Lines of Code
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
OSS	Open-Source Software
PCA	Principal Component Analysis
SATD	Self-Admitted Technical Debt
SE	Software Engineering
SZZ	Sliwerski-Zimmermann-Zeller Algorithm
TD	Technical Debt
TODO	To Be Done (Developer Comment Tag)
F1	F1-Score
ROC	Receiver Operating Characteristic
CV	Cross-Validation
API	Application Programming Interface
CFG	Control Flow Graph

Predicting High-Risk Technical Debt in Open-Source Software Projects Using Machine Learning and Code Metrics

1. INTRODUCTION

Technical debt (TD) refers to the implied cost of rework caused by choosing expedient or suboptimal solutions during software development to meet short-term goals [1]. While such choices may speed up delivery, they often reduce maintainability and increase future effort through code smells, design flaws, duplicated logic, or incomplete implementations. Over time, accumulated debt can slow development, raise defect risk, and increase the cost of change, especially in projects that evolve continuously and involve many contributors, such as open-source software (OSS).

Effectively managing TD requires not only detecting debt, but also prioritizing the items that are most likely to create serious maintenance cost, defects, or significant rework if left unresolved. In practice, manual reviews and static analysis tools often produce long lists of findings with limited guidance on what should be fixed first, making prioritization difficult under limited time and resources. Machine learning (ML) offers a data-driven way to estimate risk by learning from code metrics and repository history, and prior studies show that software attributes such as size, complexity, and change activity can support prediction models [6]. However, much of the existing work focuses on identifying whether TD exists rather than predicting which modules contain the most severe or high-risk debt. This study therefore proposes a machine learning approach to predict high-risk technical debt in OSS projects using code metrics and version history to help maintainers focus effort on the most critical parts of the system.

1.1 Problem Statement

Technical debt is an inevitable consequence of rapid software evolution, yet the fundamental challenge in practice is not simply detecting its presence, but determining which parts of a system are likely to generate substantial future maintenance burden. While static analysis tools, repository mining techniques, and comment-based detection methods can successfully identify code smells, rule violations, and self-admitted technical debt, they typically produce extensive lists of findings without providing reliable guidance on prioritization. In large and evolving systems, especially open-source projects with constrained resources and distributed contributors, maintainers cannot address all identified issues. What ultimately matters is not whether technical

debt exists, but whether certain modules are more likely to incur costly rework, repeated defect fixes, structural instability, or long-term degradation if left unaddressed. The absence of empirically grounded risk prediction mechanisms forces maintainers to rely on heuristics or subjective judgment when allocating limited refactoring effort.

The central problem addressed in this study is whether machine learning models can utilize measurable software attributes to predict modules that are likely to exhibit high maintenance risk associated with technical debt. Rather than defining high risk based solely on tool-specific severity classifications, this research conceptualizes high-risk technical debt as modules that demonstrate elevated maintenance consequences over time, such as increased future code churn, frequent bug-fix commits, instability across releases, or persistent structural complexity. By combining static code metrics with historical revision and evolution metrics, the study aims to construct a cross-project predictive framework capable of identifying modules that are statistically more likely to become maintenance-intensive. A key challenge lies in designing a labeling and validation strategy that ensures consistency across projects while capturing transferable risk signals rather than project-specific artifacts. Through cross-project and temporal validation, the research seeks to develop a generalizable model that supports proactive, data-driven technical debt prioritization.

1.2 Significance of the Study

A major practical challenge in technical debt management is prioritization. Maintainers can detect many debt indicators, yet they often lack clear guidance on which ones should be addressed first to prevent the greatest future maintenance cost. By developing a machine learning based approach for predicting high-risk technical debt at the module level, this study aims to support proactive maintenance and improve decision making in refactoring and quality assurance. The expected outcome is a prediction and ranking framework that helps maintainers focus limited effort on the modules most likely to drive costly rework, repeated defect fixes, instability, or long-term maintainability degradation if left unresolved. This can reduce avoidable maintenance effort and help prevent reliability and security exposure that may emerge when high-impact debt accumulates without timely intervention. The study will demonstrate this

practical value through empirical evaluation of prediction accuracy and prioritization effectiveness, using established measures defined in the methodology.

The study is particularly important for open-source software projects, where maintenance capacity is often constrained and work is distributed across changing contributors. In such environments, a risk-oriented prediction model can support planning and resource allocation by concentrating effort on a smaller subset of modules that is most likely to contribute disproportionately to future maintenance burden. Beyond immediate practice, this research contributes academically by shifting technical debt prediction from detection toward consequence-oriented risk prediction that aligns with real maintenance outcomes and prioritization needs. The study further provides cross-project evidence on which combinations of static code metrics and historical evolution metrics are most indicative of high-risk technical debt, and it evaluates generalizability using cross-project validation. Finally, the resulting framework can be integrated into development workflows by producing periodic risk reports or CI checks that flag modules whose predicted risk exceeds a defined threshold, enabling continuous and proactive technical debt management.

1.3 Objectives

1.3.1 General Objective

To develop and evaluate a machine learning based approach for predicting high-risk technical debt at the module level in open-source software projects using code metrics and repository history, in order to support proactive and informed technical debt prioritization.

1.3.2 Specific Objectives

To achieve the general objective, this study will:

1. To construct a module-level dataset from selected open-source software projects for cross-project analysis.
2. To extract and engineer predictive features by combining static code metrics with historical evolution metrics from repository history.

3. To train and compare machine learning models for high-risk technical debt prediction and evaluate generalizability using cross-project validation and standard metrics.
4. To identify key predictors of high-risk technical debt through feature importance and error analysis.

1.3.3 Research Questions

1. **RQ1:** How can high-risk technical debt be operationally defined and labeled in open-source projects using measurable, reproducible indicators aligned with maintenance risk?
2. **RQ2:** Which static code metrics and historical change metrics are most indicative of high-risk technical debt in software modules?
3. **RQ3:** How accurately can machine learning models predict high-risk technical debt, and which algorithms and feature sets perform best under cross-project validation?

2. RELATED LITERATURE REVIEW

This section reviews recent research on technical debt detection and prediction, focusing on machine learning approaches and the need to move from debt identification toward severity aware prioritization. It also discusses technical debt in machine learning oriented systems and recent development practices that may accelerate debt accumulation, motivating proactive and risk oriented management.

2.1 Technical Debt Detection and Classification

Early research on technical debt largely focused on identifying debt indicators in source code and related artifacts. A major line of work examines self-admitted technical debt (SATD), where developers explicitly document deferred work or shortcuts in comments, commonly using markers such as TODO or FIXME. To support large-scale analysis, researchers have developed natural language processing and machine learning methods that automatically detect SATD and, in some cases, classify the type of debt expressed. For example, DebtHunter uses machine learning to determine whether a comment indicates SATD and then categorizes the debt type, reporting improved performance compared with heuristic baselines [3]. In addition to code

comments, commit messages have also been explored as a useful signal, showing that textual features from commit history can improve prediction of technical debt related patterns [5]. Although these approaches improve automated identification and categorization, they generally treat technical debt as a detection problem and provide limited support for prioritization, since they do not explicitly estimate severity or risk in a way that helps maintainers decide what to address first.

2.2 Technical Debt Prediction Using Metrics and Project History

Beyond text based signals, a substantial body of research predicts technical debt using software metrics and repository history. These models commonly combine static code metrics, such as size and complexity, with historical change metrics, such as churn and modification frequency. Recent studies trained machine learning models to classify modules as technical debt prone using code quality metrics and reported that ensemble techniques can effectively identify TD prone modules in open source projects [6]. However, more recent work argues that relying only on local metrics may overlook broader structural and interaction effects that influence debt accumulation and persistence. To address this limitation, recent studies proposed graph based and social network style metrics derived from dependency and interaction relations among software entities, and showed that combining these metrics with traditional features can improve technical debt prediction performance [7]. Their later work further supports the value of enhanced metric sets, reporting stronger prediction effectiveness when models are evaluated under settings that better reflect real project complexity and evolution [8]. Overall, these studies indicate that both code properties and evolution signals are useful for technical debt prediction, motivating feature sets that integrate static analysis metrics with repository mining measures. At the same time, most of these approaches focus on predicting whether modules are debt prone, rather than predicting which modules contain the most severe or highest priority debt, which remains a key requirement for practical prioritization.

2.3 Technical Debt in Machine Learning and AI Oriented Systems

Technical debt characteristics can differ across domains, and machine learning oriented systems may accumulate debt more quickly due to experimentation, frequent revisions, and evolving

requirements. Recent studies reported that open source machine learning projects contain substantially higher levels of self-admitted technical debt than comparable non ML projects, suggesting that ML codebases may introduce debt earlier and carry it for longer periods [4]. Recent discussions also raise maintainability concerns related to AI assisted development tools, where AI generated code can increase duplication and reduce modularity when adopted without strong review and refactoring practices [10]. Although these trends are not the central focus of this study, they reinforce the need for severity aware prioritization so maintainers can focus on the most critical debt under limited resources.

2.4 Summary of Gaps and Motivation for This Study

Overall, recent work shows strong progress in automating technical debt detection and prediction. NLP based approaches can detect self-admitted technical debt and related signals in software artifacts [3], [5], and metric based approaches show that static code metrics and repository history can support machine learning models that identify technical debt-prone modules in open-source projects [6]. More recent studies further indicate that richer metric sets, including graph based and enhanced software metrics, can improve prediction performance beyond local code metrics alone [7], [8]. These results collectively confirm that measurable software attributes are useful for learning patterns related to technical debt and for scaling analysis across large codebases.

However, a critical gap remains in making these advances actionable for prioritization under limited maintenance capacity. Many existing approaches either focus on detecting debt indicators or predicting whether modules are debt-prone, but they provide limited guidance on which modules should be addressed first to prevent disproportionate future maintenance cost. In practice, maintainers need severity in the sense of expected consequences, such as repeated rework, defect recurrence, instability, or escalating change effort, rather than only the presence of debt indicators. In addition, when “high-risk” is defined primarily through tool-specific severity classifications, the resulting models risk learning the behavior of a particular tool rather than generalizable signals of maintenance risk that transfer across projects. As a result, there is still limited cross-project evidence on how well metric based models can predict consequence-oriented high-risk technical debt, and which combinations of static and historical

evolution metrics are most informative for that purpose. Motivated by these limitations, this study focuses on predicting high-risk technical debt at the module level using a combination of static code metrics and historical evolution metrics, while defining “high-risk” through a consistent operational policy aligned with measurable maintenance outcomes. The study emphasizes cross-project validation to assess generalizability and uses feature importance and error analysis to identify the most influential predictors of high-risk technical debt. By shifting from detection and debt-proneness toward consequence-oriented risk prediction, the proposed approach aims to provide practical support for technical debt prioritization and to contribute empirical evidence that is applicable across diverse open-source projects.

3. RESEARCH METHODOLOGY

This section presents the methodology used to achieve the research objectives and to answer the research questions. It describes the overall research design, the data sources and project selection approach, the procedures for constructing labels and extracting metrics, and the development and evaluation of the machine learning models. The methodology is designed to ensure that the proposed approach is reproducible, feasible using publicly available open-source software (OSS) data, and suitable for cross-project validation.

3.1 Research Design Overview

This study adopts an experimental research design based on empirical data from open-source software projects. The research follows a structured pipeline that mirrors the study objectives. First, high-risk technical debt is operationally defined and labeled using measurable maintenance outcome indicators observed after a fixed project snapshot. Second, module-level predictors are extracted by combining static code metrics with historical revision metrics from repository history. Third, supervised machine learning models are trained and tuned to classify modules as high-risk or not. Finally, model performance is evaluated using validation strategies that test both predictive accuracy and generalization, with emphasis on cross-project evaluation. The methodology integrates two complementary data collection approaches, static code analysis to obtain code quality and maintainability metrics, and software repository mining to obtain change and contributor activity metrics. Using these data sources together supports a more

comprehensive representation of module risk than either source alone. All experiments will be conducted using real OSS project data and reproducible scripts to strengthen the validity and repeatability of results.

3.2 Data Sources and Project Selection

This study will use empirical data from publicly available OSS projects to construct the dataset for model training and evaluation. Projects will be selected to support cross-project analysis by ensuring variation in domain and size. To reduce tooling limitations and improve comparability, the study will initially focus on projects implemented in widely supported languages such as Java or Python, where static analysis and repository mining can be applied reliably.

GitHub repositories: Candidate projects will be drawn from publicly accessible GitHub repositories with sufficient maturity and activity. Selection will prioritize projects that have an established development history, recent maintenance activity (within the last 12 months), and enough size and evolution to support reliable metric extraction (for example, at least 10,000 lines of code and 500 commits). Projects will also be chosen to remain compatible with the analysis pipeline (initially focusing on Java or Python) and to provide diversity in domain and size for cross-project evaluation. These repositories provide the source code and commit history required to compute both static code metrics and historical revision metrics.

Static analysis data (SonarQube): For each selected project, SonarQube will be used to generate static code metrics and code quality measurements. SonarQube provides metrics such as lines of code, complexity measures, duplication, test coverage where available, and counts of rule violations and code smells. Static analysis will be run on selected project snapshots such as releases or periodic commits to extract these measurements in a consistent way. In this study, SonarQube outputs will be used to produce predictor features that describe module properties and maintainability characteristics. They will not be used as the ground-truth labeling rule for high-risk technical debt.

Public technical debt datasets: To improve consistency and scale, the study will incorporate an established dataset where appropriate. The Technical Debt Dataset provides SonarQube-derived metrics and issue information for multiple OSS projects, enabling benchmarking and supporting

cross-project evaluation without requiring analysis setup for every project from scratch. Where such datasets are used, SonarQube-derived information will be treated as predictor features, while labels will be constructed from repository-based maintenance outcome indicators computed from the corresponding project history.

Supplementary project artifacts: Where feasible, additional repository artifacts such as issue trackers and commit logs will be used to support interpretation and validity discussion. For example, bug-fix commit patterns and linked issues can provide supporting evidence that modules experiencing high maintenance outcomes are also associated with repeated corrective activity. These artifacts will support contextual analysis, but they will not replace the primary labeling method.

3.3 Feature Extraction and Representation (Independent Variables)

Based on the selected projects, this study will extract a set of module-level predictors to train the machine learning models. In this study, a module refers to a source code file, and each file will be represented using a combination of static code metrics and historical change metrics. Predictive features will be organized into groups that reflect different sources of maintenance risk, including code metrics, static analysis derived maintainability metrics, process and evolution metrics, and contributor activity metrics. Table 1 summarizes the candidate feature set considered in this study.

Static code metrics: Static metrics will be extracted using SonarQube outputs and related analysis results. These features will capture size and complexity characteristics (for example, lines of code, class count, method count, cyclomatic and cognitive complexity), duplication measures (for example, duplicated code percentage), test coverage where available, and maintainability related indicators such as counts of code smells and rule violations by category. SonarQube’s technical debt or remediation estimate may be considered as a predictor where appropriate, provided it does not directly encode the labeling policy.

Historical change metrics: Historical metrics will be extracted from Git commit history using repository mining tools such as PyDriller. These features will capture evolution and contribution patterns for each file, including code churn (added and deleted lines over a defined period),

number of commits affecting the file, number of distinct contributors, file age (time since creation), time since last modification, and recent activity indicators. Where feasible, additional ownership oriented indicators may be derived (for example, the share of changes made by the dominant contributor). The intuition is that high churn, frequent edits, and shifting ownership can signal instability and increased maintenance risk.

Feature	Description	Rationale
Lines of Code (LOC)	Number of source lines in a module or file	A basic size metric that often correlates with defect and maintenance risk
Cyclomatic Complexity	Complexity count of functions or methods	High complexity can increase change effort and error-proneness
Class Count / Method Count	Number of classes and methods in a file	Structural size and design scale indicators
Duplicated Code (%)	Percentage of duplicated code	Duplication is linked to higher change effort and propagation of fixes
Test Coverage	Ratio of tested code to total code (if available)	Lower coverage can increase latent defect risk
Code Churn	Lines added and deleted over a recent window	High churn is frequently associated with instability and fault-proneness
Commit Frequency	Number of commits touching a module	Frequent changes may signal hotspot behavior
Developer Count	Number of distinct authors editing a file	High contributor count may increase coordination and ownership risk
File Age	Time since file creation and/or time since last modification	Captures maturity and recency effects in evolution

Code Smell Count	Count of maintainability related issues	Indicates maintainability degradation and potential future maintenance cost
Rule Violation Counts (by category)	Counts of issues by category (for example bugs, vulnerabilities, smells)	Provides structured quality signals as predictors, not labels
Cyclomatic Density	Complexity normalized by LOC	Captures concentration of complexity
Prior Defect Signals (optional)	Historical defect indicators where linkable	Modules with prior defects often remain fault-prone
OO Design Metrics (optional)	Coupling, cohesion, inheritance metrics where available	Classic design indicators linked to maintainability and defect risk

Table 1. Candidate predictive features for high-risk technical debt prediction.

Each module will be represented by a feature vector that combines static and historical metrics. Features will be cleaned and aligned across projects, and values will be normalized or standardized where appropriate. Collinearity will be examined, and if the feature space becomes large, feature selection or regularization will be applied to improve robustness. Dimensionality reduction methods such as PCA may be used for exploratory analysis, but final model evaluation will prioritize interpretability through feature importance analysis and error inspection. To avoid leakage, features that directly encode the labeling policy or use information from the post-snapshot observation window will not be included as input features.

3.4 High Risk Technical Debt Labeling (Dependent Variable)

The dependent variable in this study is a binary label indicating whether a module (source code file) exhibits high-risk technical debt. In this study, high-risk technical debt is defined in a consequence oriented way, referring to modules that are more likely to impose disproportionate future maintenance burden if not addressed. Therefore, labeling will be constructed using measurable maintenance outcome indicators observed after a fixed snapshot of each project, rather than using tool specific severity classifications as the ground truth.

For each selected project, a reference snapshot will be chosen at time t . All predictor features will be computed using information available up to t only, so that the model reflects realistic decision making at the time of prediction. Labels will then be derived from an observation window after t (for example, the next release or a fixed period such as the next 3 to 6 months). Within this observation window, maintenance outcome indicators will be computed per module, such as (1) the number of bug-fix commits affecting the file and (2) future code churn measured by added and deleted lines. Where feasible, defect related change signals will also be derived by linking bug-fix commits to earlier bug-inducing changes using an SZZ-style procedure.

These indicators will be combined into a maintenance risk score for each module. Modules will be labeled High-Risk TD = Yes if their score falls within a top percentile threshold within a project (for example, top 20%), and High-Risk TD = No otherwise. A percentile based policy supports comparability across projects of different sizes and activity levels and aligns the labeling with practical prioritization by focusing on the highest-risk subset of modules. To strengthen validity and maintain comparability across projects, labeling will follow a documented and reproducible policy, including snapshot selection, observation window definition, indicator computation, and threshold choice. Sensitivity analysis will be reported by varying the threshold (for example, top 10%, 20%, and 30%) to assess robustness. Where additional repository artifacts are available, labels may be cross-checked qualitatively using project history to confirm that modules labeled high risk exhibit higher maintenance activity during the observation window.

To support comparison and to clarify the added value beyond static analysis tooling, additional labeling variants may be reported as experimental baselines. For example, a severity-based proxy label derived from static analysis may be evaluated as a reference baseline, and a defect-oriented label derived from recorded defects and SZZ-style mapping may be evaluated where feasible. These variants will be used for comparison and sensitivity analysis, while the primary labeling definition remains consequence oriented and based on maintenance outcomes. The output of this step will be a labeled dataset of modules with aligned feature vectors and class labels for model training and evaluation.

3.5 Model Development, Training, and Validation

Supervised machine learning models will be trained to learn the relationship between the extracted metrics and the high-risk label. Multiple classifiers will be evaluated to identify an effective and practical baseline and to compare performance across model families. Candidate models include decision trees, random forests, support vector machines, and gradient boosted trees (for example, XGBoost or LightGBM). Boosted models are included because recent technical debt prediction research suggests that stronger learners combined with enriched metric sets can improve predictive performance [7], [8]. To ensure fair comparison, all models will be trained and tested using the same data partitions and preprocessing steps.

Model selection and tuning will be performed using k-fold cross validation on the training data. Because high-risk modules may be less frequent than low-risk modules, stratified sampling will be used, and class imbalance will be handled using class weighting and, where necessary, controlled resampling techniques. Feature selection or regularization will be applied when needed to reduce overfitting and improve generalization. Model performance will be evaluated using precision, recall, F1 score, and ROC AUC, along with confusion matrix analysis to understand error patterns. Feature importance analysis will be conducted for interpretable models and complemented by error inspection to identify which metrics most strongly contribute to high-risk predictions.

Validation will emphasize generalizability through cross-project evaluation. Cross-project validation will train models on a set of projects and test them on unseen projects, assessing whether learned patterns transfer across codebases. Where feasible, temporal validation will also be conducted by training on earlier versions of a project and testing on later versions to evaluate whether the model can forecast future high-risk modules. Finally, where additional repository artifacts are available, model outputs may be compared with qualitative signals from project history to support interpretation and discuss practical relevance.

3.6 Tools, Feasibility, Risk Mitigation, and Expected Outcomes

The following tools and libraries will be used for data collection, processing, model development, and evaluation:

Tool / Library	Purpose in this study
Python (3.9+)	Repository mining scripts, dataset construction, model training, evaluation
Git and GitHub APIs	Clone repositories, retrieve commit history, extract repository metadata
SonarQube (Community Edition) / SonarCloud	Static analysis, metric extraction, remediation estimates
SonarQube Web API	Export issues and metrics per module and per project version
PyDriller	Mine Git history, compute churn, commit frequency, file age, contributor activity
Pandas, NumPy	Cleaning, aggregation, feature engineering, dataset preparation
Scikit-learn	Baseline ML models, preprocessing, cross-validation, evaluation metrics
XGBoost / LightGBM	Gradient boosting experiments and comparison
Matplotlib, Jupyter Notebook	Exploratory analysis and plots (distributions, ROC, feature importance)

Table 2. Tools Used in the Study

The proposed approach is feasible because it relies on publicly available OSS repositories and established, widely used tools for static analysis and repository mining. In addition, an existing technical debt dataset can be used to support scale and benchmarking, reducing the effort required to prepare data from scratch [2]. The main risks include (1) static analysis failures due to build or dependency issues in some projects, (2) class imbalance if high-risk modules are relatively rare, (3) potential inconsistency across projects in how metrics are computed due to

differing build configurations and analysis settings, and (4) labeling limitations due to noise in repository-based maintenance outcome indicators (for example, imperfect identification of bug-fix commits, incomplete linking between issues and commits, or limitations of SZZ-style mapping where applied). These risks will be mitigated through careful project selection with clear inclusion criteria, fallback to alternative projects if analysis fails, consistent analysis configuration across projects, documented snapshot and labeling rules, and manual inspection of a small sample of labeled modules to validate that labels align with the intended notion of high maintenance risk. Class imbalance will be handled using stratified sampling and class weighting, with resampling applied only when necessary. Cross-project validation will be emphasized to reduce project-specific bias and to assess whether learned patterns generalize beyond the training projects [6], [7], [8].

The expected outputs of this study include: (1) a reproducible dataset of OSS modules represented by static and historical metrics and labeled as high-risk or not using maintenance outcome indicators observed after a snapshot (for example, post-snapshot bug-fix activity and code churn), (2) a comparative evaluation of supervised learning models for predicting high-risk technical debt under cross-project settings, and (3) empirical insights into which metrics most strongly contribute to high-risk predictions. Model effectiveness will be assessed using precision, recall, F1 score, and ROC AUC, supported by confusion matrix analysis. Practical usefulness will be assessed by the ability of the models to prioritize a smaller subset of modules that contains a large share of high-risk labels, supporting maintenance planning under limited resources. The study will also discuss how the approach could be integrated into development workflows, for example by producing periodic risk reports or CI checks that highlight modules whose predicted risk exceeds a defined threshold for review.

4. WORK SCHEDULE AND COST BREAKDOWN

4.1 Work Schedule and Timeline

Each activity reflects a core stage in the research methodology, including technical tasks such as data mining, model training, and evaluation. The schedule ensures time for iterative refinement and thesis documentation.

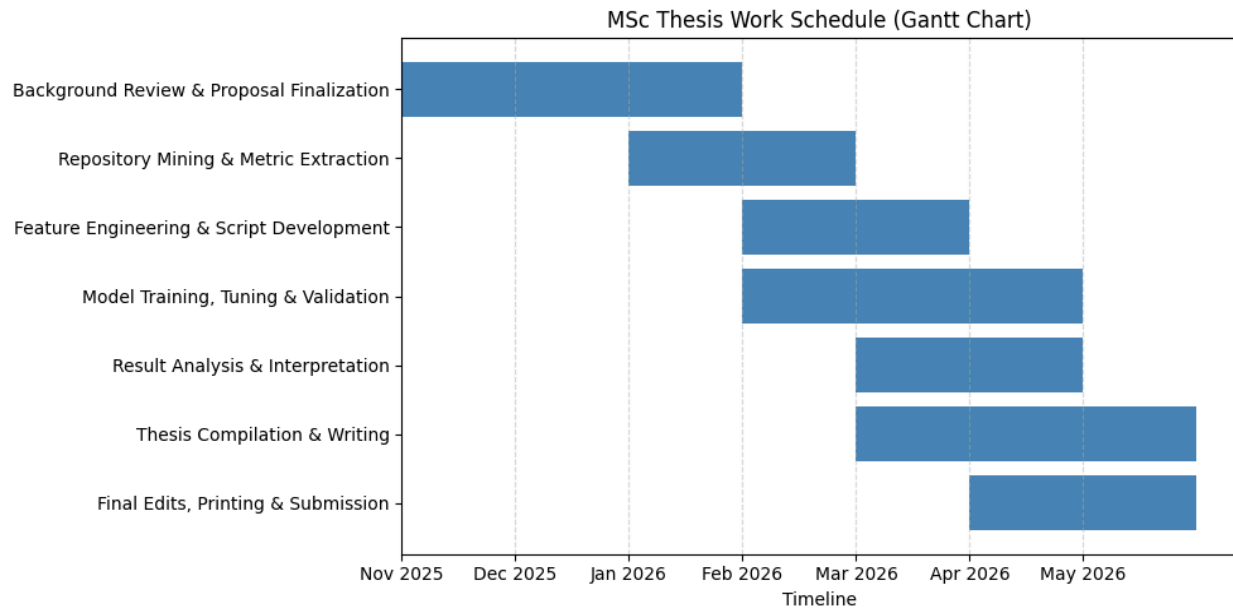


Figure 1. Thesis Work Schedule (Gantt Chart)

No.	Activity	Start Date	End Date
1	Background Review & Proposal Finalization	Nov 2025	Jan 2026
2	Repository Mining & Metric Extraction	Jan 2026	Feb 2026
3	Feature Engineering & Script Development	Feb 2026	Mar 2026
4	Model Training, Tuning & Validation	Feb 2026	Apr 2026
5	Result Analysis & Interpretation	Mar 2026	Apr 2026
6	Thesis Compilation & Writing	Mar 2026	May 2026
7	Final Edits, Printing & Submission	Apr 2026	May 2026

Table 3. Activity Breakdown

4.2 Budget Breakdown

The following table summarizes the estimated costs required for successful completion of the thesis. The total projected budget is ETB 19,800.

No.	Item Description	Unit	Quantity	Unit Price (ETB)	Total Cost (ETB)
1	Internet Data (for research communication and online work)	Month	4	1,700	6,800
2	Printing & Binding of Thesis (final document production)	Copies	–	–	9,000
3	Miscellaneous Expenses (additional supplies, software, etc.)	–	–	–	4,000
Total	<i>Estimated Total Budget</i>				19,800

Table 4. Budget Breakdown

REFERENCES

- [1] W. Cunningham, “The WyCash portfolio management system,” Proc. OOPSLA, Vancouver, BC, Canada, Oct. 1992, pp. 29–30.
- [2] V. Lenarduzzi, D. Taibi, N. Saarimäki, and D. A. Tamburri, “The technical debt dataset,” Proc. 16th Int. Conf. Mining Softw. Repositories (MSR), Montreal, QC, Canada, May 2019, pp. 327–330.
- [3] M. Sala, G. Scanniello, C. Gravino, and G. Tortora, “DebtHunter: A machine learning-based approach for self-admitted technical debt detection,” J. Syst. Softw., vol. 171, p. 110829, Feb. 2021.
- [4] S. Bhatia, C. Arora, and M. Sabetzadeh, “Self-admitted technical debt in machine learning systems,” Empir. Softw. Eng., vol. 28, no. 4, Art. no. 89, Jul. 2023.
- [5] L. Rantala and M. V. Mäntylä, “Predicting technical debt from commit messages using machine learning,” Inf. Softw. Technol., vol. 122, p. 106269, Jan. 2020.
- [6] D. Tsoukalas, N. Mittas, and L. Angelis, “Identifying technical debt-prone software modules using machine learning techniques,” J. Syst. Softw., vol. 165, p. 110567, Nov. 2020.
- [7] Z. Jiang, T. Chen, and Y. Zhou, “Improving technical debt prediction with graph-based and social-network metrics,” Empir. Softw. Eng., vol. 29, no. 1, Art. no. 12, Jan. 2024.
- [8] Z. Jiang, Z. Li, T. Chen, and Y. Zhou, “Towards more effective technical debt prediction: Leveraging enhanced software metrics,” Inf. Softw. Technol., vol. 167, p. 107332, Jan. 2025.
- [9] SonarSource, “SonarQube documentation: Technical debt and code quality model; Rule severity levels and remediation cost,” 2024. [Online]. Available: <https://docs.sonarsource.com/>
- [10] B. Doerrfeld, “How AI-generated code compounds technical debt,” LeadDev, Feb. 2025. [Online]. Available: <https://leaddev.com/>